

LEXICAL ANALYZER

A **lexical analyzer** (also called a **lexer** or **scanner**) is the first phase of a compiler or interpreter. It reads source code character-by-character and converts it into meaningful **tokens**.

What Does It Do?

The lexical analyzer breaks down raw source code into recognizable units called **tokens**, each with a type and value. For example:

Eg:

```
int x = 5;
```

Gets tokenized as:

- `int` → **Keyword**
- `x` → **Identifier**
- `=` → **Operator**
- `5` → **Integer**
- `;` → **Operator/Delimiter**

Why Is It Important?

1. **Removes noise** — Ignores whitespace, comments (in a real lexer)
2. **Recognizes patterns** — Identifies keywords, numbers, strings, identifiers
3. **Error detection** — Catches invalid characters early
4. **Prepares for parsing** — The next compiler phase (parser) works with tokens, not raw text

How This Program Works

Your program uses a **two-pointer sliding window** approach:

1. **Left pointer** — marks the start of a token
2. **Right pointer** — scans forward until it hits a delimiter
3. **Extract substring** — between left and right
4. **Classify** — check if it's a keyword, number, identifier, etc.
5. **Output token** — print type and value

Example:

Step-by-step for "float pi = 3.14":

Step	Substring	Classification	Output
1	float	Keyword (matches keyword list)	Token: Keyword, Value: float
2	pi	Valid identifier (starts with letter)	Token: Identifier, Value: pi
3	=	Single operator	Token: Operator, Value: =
4	3.14	Real number (contains one dot)	Token: Real Number, Value: 3.14

Token Types Recognized

Your program identifies:

- **Keywords** — int, float, if, while, return, etc.
- **Identifiers** — variable/function names (x, myVar, _count)
- **Operators** — +, -, *, /, =, >=, ==, etc.
- **Numbers** — integers (42) and decimals (3.14)
- **Literals** — strings ("hello") and characters ('a')
- **Delimiters** — ;, ,, (,), {, }, etc.

Real-World Application

Lexical analyzers are used in:

- **Compilers** (C, Java, Python) — first phase before parsing
- **Interpreters** — to understand code before executing
- **Code editors** — syntax highlighting uses lexers
- **Linters** — style checkers that analyze code structure

SOURCE CODE:

```
#include<ctype.h>

#include<stdbool.h>

#include<stdio.h>

#include<stdlib.h>

#include<string.h>

#define MAX_LENGTH 256

bool isDelimiter(char chr) {

    return (chr == ' ' || chr == '+' || chr == '-' ||

            chr == '*' || chr == '/' || chr == ',' ||

            chr == ';' || chr == '%' || chr == '>' ||

            chr == '<' || chr == '=' || chr == '(' ||

            chr == ')' || chr == '[' || chr == ']' ||

            chr == '{' || chr == '}');

}

bool isOperator(char chr) {

    return (chr == '+' || chr == '-' || chr == '*' ||

            chr == '/' || chr == '>' || chr == '<' ||

            chr == '=');

}

bool isMultiOperator(char* str) {

    return (strcmp(str, "==") == 0 || strcmp(str, "<=") == 0 ||

            strcmp(str, ">=") == 0 || strcmp(str, "!=") == 0);

}

bool isValidIdentifier(char* str) {

    if (!isalpha(str[0]) && str[0] != '_')

    {
```

```

    return false;
}
int i;
for ( i = 1; str[i]; i++)
{
    if (!isalnum(str[i]) && str[i] != '_') return false;
}
return true;
}

bool isKeyword(char* str) {
    const char* keywords[] = {
        "auto","break","case","char","const","continue","default","do",
        "double","else","enum","extern","float","for","goto","if",
        "int","long","register","return","short","signed","sizeof","static",
        "struct","switch","typedef","union","unsigned","void","volatile","while"
    };
    int i;
    for ( i = 0; i < sizeof(keywords)/sizeof(keywords[0]); i++) {
        if (strcmp(str, keywords[i]) == 0) return true;
    }
    return false;
}

bool isInteger(char* str) {
    if (str == NULL || *str == '\0') return false;
    int i = 0;
    while (isdigit(str[i])) i++;
    return str[i] == '\0';
}

bool isRealNumber(char* str) {

```

```

int i = 0, dotCount = 0;
if (str == NULL || *str == '\0') return false;
while (str[i]) {
    if (str[i] == '.') {
        dotCount++;
        if (dotCount > 1) return false;
    } else if (!isdigit(str[i])) {
        return false;
    }
    i++;
}
return dotCount == 1;
}

bool isStringLiteral(char* str) {
    return (str[0] == '"' && str[strlen(str)-1] == '"');
}

bool isCharLiteral(char* str) {
    return (str[0] == '"' && str[strlen(str)-1] == '"' && strlen(str) == 3);
}

char* getSubstring(char* str, int start, int end) {
    int subLength = end - start + 1;
    char* subStr = (char*)malloc((subLength + 1) * sizeof(char));
    strncpy(subStr, str + start, subLength);
    subStr[subLength] = '\0';
    return subStr;
}

int lexicalAnalyzer(char* input) {
    int left = 0, right = 0;
    int len = strlen(input);

```

```

while (right <= len && left <= right) {
    if (!isDelimiter(input[right]))
        right++;
    if (isDelimiter(input[right]) && left == right) {
        if (isOperator(input[right]))
            printf("Token: Operator, Value: %c\n", input[right]);
        right++;
        left = right;
    }
    else if (isDelimiter(input[right]) && left != right
        || (right == len && left != right)) {
        char* subStr = getSubstring(input, left, right - 1);
        if (isKeyword(subStr))
            printf("Token: Keyword, Value: %s\n", subStr);
        else if (isMultiOperator(subStr))
            printf("Token: Operator, Value: %s\n", subStr);
        else if (isInteger(subStr))
            printf("Token: Integer, Value: %s\n", subStr);
        else if (isRealNumber(subStr))
            printf("Token: Real Number, Value: %s\n", subStr);
        else if (isStringLiteral(subStr))
            printf("Token: String Literal, Value: %s\n", subStr);
        else if (isCharLiteral(subStr))
            printf("Token: Char Literal, Value: %s\n", subStr);
        else if (isValidIdentifier(subStr))
            printf("Token: Identifier, Value: %s\n", subStr);
        else
            printf("Token: Unidentified, Value: %s\n", subStr);
        free(subStr);
    }
}

```

```

        left = right;
    }
}
return 0;
}

void stripNewline(char* str) {
    size_t len = strlen(str);
    while (len > 0 && (str[len - 1] == '\n' || str[len - 1] == '\r')) {
        str[--len] = '\0';
    }
}

int main() {
    char lex_input[MAX_LENGTH];
    printf("Enter a line of expression to analyze (max %d chars):\n", MAX_LENGTH - 1);
    if (fgets(lex_input, sizeof(lex_input), stdin) == NULL) {
        fprintf(stderr, "Error reading input.\n");
        return 1;
    }
    stripNewline(lex_input);
    if (strlen(lex_input) == 0) {
        printf("No input provided.\n");
        return 0;
    }
    printf("\nFor Expression \"%s\":\n", lex_input);
    lexicalAnalyzer(lex_input);
    return 0;
}

```

OUTPUT:

Enter a line of expression to analyze (max 255 chars):

```
float pi = 3.14; if (pi >= 3) return "valid";
```

For Expression "float pi = 3.14; if (pi >= 3) return "valid";":

Token: Keyword, Value: float

Token: Identifier, Value: pi

Token: Operator, Value: =

Token: Real Number, Value: 3.14

Token: Keyword, Value: if

Token: Identifier, Value: pi

Token: Operator, Value: >

Token: Operator, Value: =

Token: Integer, Value: 3

Token: Keyword, Value: return

Token: String Literal, Value: "valid"